e.l.f. Beauty Brewery API

Technical Documentation | C# Interview Assessment

This .NET 8 Web API exposes open brewery data from Open Brewery DB with search, sorting, caching, API versioning, authentication, logging, SQLite storage, and automated tests.

1. Architecture

- **Core:** domain models, exceptions, and interfaces.
- **Infrastructure:** typed HTTP client, source-to-domain mapper, cache, in-memory repository, SQLite/EF Core repository, and application service.
- **API:** versioned controllers, authentication, Swagger, middleware, configuration, and logging.
- **Tests:** unit, HTTP client, validation, and WebApplicationFactory integration tests.

The design applies SOLID principles: controllers depend on the service contract, services depend on repository/client/cache abstractions, and storage implementations are replaceable through dependency injection.

2. Acceptance Criteria

Requirement	Implementation
List brewery name, city, phone	`GET /api/v1/breweries`
In-memory storage	Default `InMemoryBreweryRepository`
Search	`search` across name, city, and state
Sorting	`sortBy=name|city|distance`, with `asc|desc`
Distance calculation	Haversine calculation with validated coordinates
Mapping	Open Brewery DB DTO mapped to generic Brewery model

10-minute cache	Memory cache with stale-while-revalidate after expiry
Error handling	ProblemDetails middleware with status and traceId

3. Bonus Features

- Autocomplete: `GET /api/v1/breweries/autocomplete?query=do`
- API versioning: `/api/v1/...`
- Serilog console and rolling-file logging
- SQLite + EF Core selectable with `Storage:Provider=Sqlite`
- API key authentication using `X-Api-Key`

4. Scalability Decisions

- External source loading is bounded by configurable `MaxPages` and `PageSize`.
- SQLite performs filtering, sorting, count, and paging in the database using EF Core.
- Expired cache data is served immediately while a single background refresh runs.
- Pagination protects against large offsets without integer overflow.
- In-memory mode remains bounded for the assignment and is replaceable by SQLite.

5. API Usage

Configure a local API key with:

```
cd src\ElfBeauty.BreweryApi.Api
dotnet user-secrets init
dotnet user-secrets set "ApiKeySettings:ApiKey" "your-dev-key"
dotnet run
```

Open Swagger at `http://localhost:5000/swagger`, authorize with the same key, and test:

```
GET /health
GET /api/v1/breweries?search=dog&sortBy=name&sortDirection=asc
GET /api/v1/breweries?
sortBy=distance&latitude=39.7392&longitude=-104.9903
GET /api/v1/breweries/autocomplete?query=do&maxResults=10
```

6. Quality Evidence

Build: 0 warnings, 0 errors

Automated tests: 37 passed, 0 failed

Tests cover mapping, coordinates, distance, search, all sorting modes, autocomplete, caching behavior, repository query behavior, external API failures, malformed JSON, authentication, validation, and end-to-end API flows.

7. Repository

https://github.com/mohdbhura1202/elf-beauty-brewery-api

No API keys, passwords, local databases, build artifacts, or user secrets are committed to the repository.